

Interpolación Geométrica de Contornos de Gliomas entre Slices de Resonancia Magnética

Abel Albuez, Santiago Gil Gallego y Jesús Alberto

Maestría en Ingeniería de Sistemas y Computación

Pontificia Universidad Javeriana, Bogotá D.C., Colombia

Profesor: Leonardo Flórez-Valencia Geometría Computacional 2026

Repositorio: <https://github.com/AbelAlbuez/computational-geometry-cgal>

Resumen—Se presenta un pipeline completo de interpolación geométrica de contornos del tumor activo (*Enhancing Tumor*, ET) y reconstrucción tridimensional, sobre el dataset público *BraTS 2024 GLI*. A partir de los contornos 2-D extraídos por corte axial se implementan tres métodos de interpolación en C++17/20 sobre CGAL: (M1) lineal con correspondencia de vértices y resolución de auto-intersecciones mediante Bentley–Ottmann; (M2) spline cúbico natural a lo largo del eje z ; y (M3) basado en forma mediante campos de distancia con signo (con conexión Voronoi / eje medio). La pila densificada se cierra en una superficie 3-D con reconstrucción de *Poisson*, y se calculan métricas de volumen, área, simetría y curvatura. La calidad de la interpolación se evalúa con un protocolo *dejar-un-corte-fuera* sobre 110 casos (5 190 cortes ocultos por método), comparando contra la verdad de terreno. Los métodos lineal y basado en forma resultan los más fiables en datos reales (Dice mediana $\approx 0,92$). El módulo de interpolación lineal y la resolución de auto-intersecciones fueron desarrollados por Abel Albuez; los métodos spline y basado en forma, la reconstrucción, las métricas y el visualizador completan el sistema del equipo.

Index Terms—interpolación de contornos, geometría computacional, CGAL, Bentley–Ottmann, BraTS, resonancia magnética, gliomas

I. INTRODUCCIÓN

La reconstrucción tridimensional de tumores cerebrales a partir de secuencias de resonancia magnética (MRI) es una tarea relevante para el diagnóstico, la planeación quirúrgica y el seguimiento clínico de pacientes con gliomas. Una estrategia extendida consiste en segmentar slice por slice la región de interés y luego *interpol* los contornos 2D obtenidos para reconstruir una superficie cerrada en 3D.

Este trabajo utiliza el dataset *BraTS 2024 GLI* [4] (Brain Tumor Segmentation Challenge — Adult Glioma), que provee volúmenes de MRI con segmentaciones expertas. De cada volumen se extrae el contorno del *Enhancing Tumor* (label ET = 3) por slice axial mediante *scikit-image*, obteniendo polígonos cerrados almacenados en formato *.obj*.

La motivación última proviene de la línea de investigación del autor sobre segmentación automática de gliomas: una vez segmentado el tumor en slices 2D, se requiere un módulo geométrico que produzca slices intermedios consistentes — sin auto-intersecciones — para alimentar etapas posteriores de reconstrucción volumétrica.

Este informe extiende dicho módulo a un **pipeline completo**: además de la interpolación lineal (M1), se incorporan

un método por *spline* en z (M2) y uno basado en forma por campo de distancia (M3); la pila de contornos se cierra en 3-D con reconstrucción de *Poisson* y se valida cuantitativamente con un protocolo *dejar-un-corte-fuera* sobre 110 casos reales. La Sección III detalla el módulo lineal; las Secciones IV–VIII describen los métodos adicionales, la reconstrucción, las métricas y la evaluación.

II. MARCO TEÓRICO

II-A. Interpolación entre slices paralelos

Barequet y Sharir [1] formalizan el problema de interpolación lineal por tramos entre cortes poligonales paralelos como un problema de *correspondencia* entre vértices: dadas dos poligonales A y B en planos paralelos, construir una superficie triangulada que las una. Su trabajo establece el marco bajo el cual opera el módulo aquí descrito, restringido al caso $|A| = |B|$ (igualando los vértices mediante *resampling*) y a la interpolación lineal directa de vértices correspondientes.

II-B. Interpolación en imágenes médicas

Lehmann *et al.* [2] presentan un estudio comparativo de métodos de interpolación de *intensidad* en imágenes médicas (vecino más cercano, lineal, B-splines, cúbica, Lagrange, entre otros), concluyendo que los B-splines de alto grado ofrecen el menor error de interpolación. Conviene distinguir ese problema —la interpolación de valores de intensidad entre vóxeles— de la interpolación de *contornos/forma* que aborda este trabajo, para la cual las referencias son los métodos basados en forma (Sección IV-B) [8], [9]. La interpolación lineal por vértices se adopta aquí como *baseline* por su simplicidad y bajo costo, antes de considerar las variantes basadas en spline (M2) y en forma (M3).

II-C. Polígonos auto-intersecantes

Souviron [3] presenta un esquema de detección y corrección de auto-intersecciones en polígonos cerrados con memoria mínima. En este trabajo se adopta una estrategia equivalente en el espíritu pero apoyada en CGAL: se detectan los cruces con el barrido de Bentley–Ottmann y se descompone el polígono en sub-lazos, reteniendo el de mayor área. El algoritmo de Bentley–Ottmann en sí está implementado dentro de la librería *pujCGAL* provista por el profesor del curso.

III. METODOLOGÍA — MÓDULO DE ABEL

El módulo está estructurado como una cadena de clases independientes sobre el kernel CGAL::Exact_predicates_inexact_constructions y el tipo TContour = std::vector<Point_2>. Las responsabilidades se descomponen en cuatro etapas, cada una encapsulada en una clase con un único método estático.

III-A. Resampling por longitud de arco (ContourResampler)

Dado un contorno cerrado $C = \{p_0, p_1, \dots, p_{m-1}\}$, se calcula la longitud de arco acumulada

$$s_0 = 0, \quad s_{i+1} = s_i + \|p_{(i+1) \bmod m} - p_i\|, \quad i = 0, \dots, m-1, \quad (1)$$

con $L = s_m$ la longitud total del contorno cerrado.

Para producir un contorno con exactamente n vértices uniformemente distribuidos por longitud de arco se evalúa, para cada $k \in \{0, \dots, n-1\}$, el parámetro $s_k^* = kL/n$, se localiza el segmento $[p_i, p_{i+1}]$ tal que $s_i \leq s_k^* \leq s_{i+1}$, y se interpola

$$u = \frac{s_k^* - s_i}{s_{i+1} - s_i}, \quad q_k = (1 - u)p_i + up_{(i+1) \bmod m}. \quad (2)$$

Esta etapa garantiza la precondition $|A| = |B|$ requerida por la interpolación lineal.

III-B. Interpolación lineal (LinearInterpolator)

Sean $A = \{a_0, \dots, a_{n-1}\}$ y $B = \{b_0, \dots, b_{n-1}\}$ dos contornos con el mismo número de vértices (tras el resampling). El contorno interpolado $P(t)$ para $t \in [0, 1]$ se define componente a componente:

$$P_i(t) = (1 - t)a_i + tb_i, \quad i = 0, 1, \dots, n-1. \quad (3)$$

En particular, $t = 0,5$ produce el slice intermedio entre A y B .

```
1 for( std::size_t i = 0; i < n; ++i )
2 {
3     const double ax = CGAL::to_double( A[i].x() );
4     const double ay = CGAL::to_double( A[i].y() );
5     const double bx = CGAL::to_double( B[i].x() );
6     const double by = CGAL::to_double( B[i].y() );
7     out.emplace_back( (1.0 - t)*ax + t*bx,
8                       (1.0 - t)*ay + t*by );
9 }
```

Listing 1. Núcleo de la interpolación lineal.

III-C. Detección de auto-intersecciones (SelfIntersectionResolver)

El contorno interpolado se convierte primero en un vector de segmentos CGAL conectando vértices consecutivos en orden circular:

$$S_i = \overline{P_i P_{(i+1) \bmod n}}, \quad i = 0, \dots, n-1. \quad (4)$$

A continuación se invoca el algoritmo de barrido Bentley–Ottmann [10] provisto por pujCGAL:

```
std::vector< TPoint > inters;
pujCGAL::SegmentsIntersection::BentleyOttmann(
    segs.begin(), segs.end(),
    std::back_inserter( inters )
) kernel
```

Listing 2. Invocación del barrido Bentley–Ottmann de pujCGAL.

Se filtran las intersecciones que coinciden con un vértice original del contorno (extremos compartidos entre aristas consecutivas, que no representan cruces reales). Si el conjunto resultante es vacío, el contorno se retorna inalterado.

III-D. Resolución de cruces

Cuando existen cruces, cada punto de intersección p_j se asocia a las dos aristas que lo contienen y se calcula el parámetro $t \in [0, 1]$ de su posición a lo largo de cada arista. Esto permite construir una *traversal subdividida*: una lista ordenada que recorre el contorno insertando los puntos de cruce en su posición correspondiente.

Sobre esta traversal se aplica un esquema clásico de extracción de lazos mediante pila: cuando un punto de intersección aparece por segunda vez se cierra un sub-lazo con todos los nodos apilados desde su primera aparición. Cada sub-lazo L_k tiene un área absoluta dada por la fórmula del polígono (shoelace):

$$\text{Area}(L_k) = \frac{1}{2} \left| \sum_{i=0}^{|L_k|-1} (x_i y_{i+1} - x_{i+1} y_i) \right|. \quad (5)$$

El módulo retiene el sub-lazo L_{k^*} con $k^* = \arg \max_k \text{Area}(L_k)$, es decir, el lazo exterior de mayor extensión, descartando lazos parásitos típicos cuando los contornos fuente difieren mucho en forma.

IV. MÉTODOS ADICIONALES DE INTERPOLACIÓN

El módulo lineal de la Sección III (en adelante **M1**) se complementó con dos métodos que comparten el mismo tipo TContour y kernel: un método **M2** basado en *splines* a lo largo del eje z y un método **M3** basado en forma mediante campos de distancia con signo.

IV-A. M2 — Spline cúbico natural / polinomial en z

M1 sólo emplea dos cortes y es C^0 en z (pila escalonada). **M2** usa una ventana de $m \geq 3$ cortes $z_0 < \dots < z_{m-1}$: encadenando la correspondencia cíclica entre cortes consecutivos (Sección VII-B3), cada índice de vértice i traza una trayectoria $\gamma_i(z) = (x_i(z), y_i(z))$. Cada coordenada se ajusta de forma independiente con un *spline cúbico natural* C^2 , cuyas segundas derivadas M_k resuelven el sistema tridiagonal

$$h_{k-1}M_{k-1} + 2(h_{k-1} + h_k)M_k + h_kM_{k+1} = 6 \left(\frac{y_{k+1} - y_k}{h_k} - \frac{y_k - y_{k-1}}{h_{k-1}} \right), \quad (6)$$

con $h_k = z_{k+1} - z_k$ y condiciones naturales $M_0 = M_{m-1} = 0$, resuelto en $O(m)$ por el algoritmo de Thomas [9]. La variante *polinomial* ajusta por mínimos cuadrados un polinomio de grado $d = \min(3, m-1)$ por coordenada; se incluye como contraste didáctico, pues sub-ajusta u oscila en ventanas largas.



Figura 1. Reconstrucción 3-D del tumor *BraTS-GLI-00528-101*. De izquierda a derecha: verdad de terreno (vóxeles), lineal, spline y SDF. Las tres reconstrucciones son cerradas y reproducen la forma global; la verdad de terreno es más “escalonada” por provenir de la máscara de vóxeles.

IV-B. M3 — Basado en forma (campo de distancia con signo)

M3 no mueve vértices sino la representación implícita del contorno, en la línea de la interpolación basada en forma [8]. Cada contorno se convierte en un campo de distancia con signo

$$\Phi_C(\mathbf{q}) = \pm \min_{\mathbf{c} \in \partial C} \|\mathbf{q} - \mathbf{c}\|, \quad (7)$$

positivo dentro del tumor. La distancia se evalúa con un árbol AABB de CGAL sobre las aristas del contorno y el signo con `Polygon_2::bounded_side`. El mapa $\mathbf{q} \mapsto \min_{\mathbf{c}} \|\mathbf{q} - \mathbf{c}\|$ es exactamente la distancia al contorno: la partición del plano en regiones de “arista más cercana” es el **diagrama de Voronoi** de las aristas y el lugar donde el punto más cercano no es único es el **eje medio** de la forma [11]. Los dos campos se mezclan linealmente y el contorno intermedio es el conjunto de nivel cero, extraído con *marching squares*:

$$\Phi_t = (1 - t) \Phi_A + t \Phi_B, \quad C_t = \{\mathbf{q} : \Phi_t(\mathbf{q}) = 0\}. \quad (8)$$

Antes de mezclar se aplica una pre-alineación rígida (centroide y ejes principales). M3 es robusto a grandes desplazamientos y a cambios de topología, a costa de operar sobre una rejilla.

V. RECONSTRUCCIÓN 3-D POR SUPERFICIE DE POISSON

La pila densificada (cortes originales más los intermedios de M1/M2/M3) se ensambla en una nube de puntos orientada: a cada vértice (x, y) del corte k se le asigna la coordenada $(x, y, k \Delta z)$, con Δz leído de la cabecera NIFTI. La normal exterior en el plano se obtiene girando la tangente unitaria -90° (contorno CCW); se refina con `jet_estimate_normals` y se orienta globalmente con `mst_orient_normals`. La superficie cerrada se obtiene con `poisson_surface_reconstruction_delaunay`, que construye una triangulación de Delaunay 3-D del conjunto y resuelve una función indicadora cuyo gradiente ajusta las normales [5]. El volumen se calcula por el teorema de la divergencia,

$$V = \frac{1}{6} \sum_{\Delta} \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}). \quad (9)$$

La Figura 1 muestra la superficie reconstruida por los tres métodos junto a la verdad de terreno (superficie por *marching cubes* [6] de los vóxeles segmentados).

VI. MÉTRICAS DE EVALUACIÓN

Sobre cada superficie se calculan: *volumen* (en mm^3 , tras `orient_to_bound_a_volume`) y *área*; un chequeo cruzado del volumen por regla trapezoidal sobre las áreas *shoelace* de la pila; un *score de simetría* reflectiva (ejes principales del tensor de inercia, Dice volumétrico con `Side_of_triangle_mesh` y Hausdorff simétrica contra la malla reflejada); y la *curvatura media* discreta por Laplaciano cotangente [7] (implementada a mano, pues no existe en CGAL 5.6). La exactitud de la interpolación se mide con Dice $= \frac{2|X \cap Y|}{|X| + |Y|}$, IoU, Hausdorff simétrica y error relativo de área.

VII. RESULTADOS CUALITATIVOS DEL MÓDULO LINEAL (M1)

VII-A. Configuración experimental

La evaluación se realizó sobre el dataset *BraTS 2024 GLI* restringido al label *Enhancing Tumor* (ET, valor 3). Los contornos se extrajeron por slice axial mediante `scikit-image` y se serializaron en formato `.obj` 2D. Sobre este corpus se ejecutaron **4 pares representativos** de slices con parámetro de interpolación fijo $t = 0,5$, cada uno con una intención distinta:

1. slices consecutivos del mismo caso (variación morfológica mínima);
2. slices separados por 6 cortes del mismo caso (variación moderada);
3. par extremo (primer y último slice con ET) dentro de un mismo caso (variación máxima intra-caso);
4. par cruzado entre dos casos distintos (primer slice del caso X contra primer slice del caso Y).

Sobre la versión inicial del interpolador se incorporaron **tres mejoras geométricas** de pre-alineación (orientación, centroide y rotación cíclica óptima del punto de inicio), descritas en la Sección VII-B. Cada ejecución produce un `.obj` 3D y una visualización PNG con los dos contornos fuente y el contorno interpolado superpuestos.

VII-B. Mejoras geométricas implementadas

VII-B1. Normalización de orientación (CCW): La interpolación lineal vértice a vértice es *sensible al sentido de recorrido*: si A está en sentido CCW y B en CW, la correspondencia $A_i \leftrightarrow B_i$ recorre los contornos en direcciones opuestas y produce un polígono retorcido. Para evitarlo se calcula el área con signo de cada contorno mediante la fórmula *shoelace*:

$$\text{area}(C) = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i), \quad (10)$$

y si $\text{area}(C) < 0$ se invierte el orden de los vértices, garantizando que ambos contornos sean CCW antes de interpolar.

VII-B2. Alineación por centroide: La interpolación lineal implícitamente promedia las posiciones absolutas. Para que la geometría intermedia no quede distorsionada por una traslación entre A y B , se calculan los centroides

$$\bar{c}_A = \frac{1}{|A|} \sum_i A_i, \quad \bar{c}_B = \frac{1}{|B|} \sum_i B_i, \quad (11)$$

Cuadro I
RESULTADOS DE LOS 4 PARES REPRESENTATIVOS ($t = 0,5$) TRAS
APLICAR LAS MEJORAS DE PRE-ALINEACIÓN.

Caso	A→B	A	B	Resam.	k^*	Auto-int.
00008-100 (consecutivos)	0070 → 0071	32	30	32	0	No
00008-100 (separados 6)	0066 → 0072	14	24	24	1	No
00008-101 (extremos)	0070 → 0072	26	16	26	2	No
100 vs 101 (cruzado)	0066 → 0070	14	26	26	24	No

se trasladan ambos contornos al origen, se interpola en el sistema centrado y el contorno resultante se traslada al centroide promedio $\bar{c}_M = \frac{1}{2}(\bar{c}_A + \bar{c}_B)$.

VII-B3. Rotación óptima del punto de inicio: Aún con $|A| = |B|$ tras el *resampling*, el primer vértice de cada contorno es arbitrario. Una correspondencia $A_i \leftrightarrow B_i$ desalineada estira innecesariamente las aristas del polígono interpolado. Se busca el desplazamiento cíclico k^* que minimiza la suma de distancias al cuadrado:

$$k^* = \arg \min_{k \in \{0, \dots, n-1\}} \sum_{i=0}^{n-1} \|A_i - B_{(i+k) \bmod n}\|^2, \quad (12)$$

y se aplica $B \leftarrow \text{rotate}(B, k^*)$ antes de interpolar.

VII-C. Tabla de resultados

La Tabla I reporta los conteos de vértices, la rotación óptima detectada k^* y el estado de auto-intersección para los cuatro pares. La columna *Resam.* indica el número común de vértices tras el *resampling*, igual a $\max(|A|, |B|)$.

En los cuatro casos el barrido de Bentley–Ottmann no reportó auto-intersecciones, por lo que el `SelfIntersectionResolver` retornó el contorno original sin modificación. Los cuatro contornos interpolados resultantes son polígonos simples válidos. La salida cruda del ejecutable para el primer par ilustra el formato textual:

```
Contour A: 32 vertices
Contour B: 30 vertices
Resampled to: 32 vertices
Optimal rotation of B: 0
Interpolated contour: 32 vertices
Self-intersections detected: no
Result written to output/casol_consecutivos/
contour_interpolated.obj
```

Listing 3. Salida de `contour_interpolator` sobre dos slices consecutivos.

VII-D. Análisis por caso

Caso 1 — consecutivos (0070 → 0071, $k^* = 0$). Resultado correcto: el contorno interpolado queda visiblemente entre A y B y conserva la forma general del tumor. La rotación óptima nula indica que las parametrizaciones por longitud de arco ya estaban naturalmente alineadas; persiste un artefacto leve en la zona derecha por la diferencia de forma local entre los dos slices.

Caso 2 — separados 6 cortes (0066 → 0072, $k^* = 1$). Los contornos A y B ocupan zonas distintas del plano xy . La alineación por centroide es la que más aporta aquí: sin ella, el interpolado quedaría sesgado hacia el contorno mayor.

Con ella, el resultado se ubica en el centroide promedio $\bar{c}_M$ y es geoméricamente válido, aunque sin correspondencia anatómica directa con un slice z intermedio.

Caso 3 — extremos del mismo caso (0070 → 0072, $k^* = 2$). Mejor resultado cualitativo. La rotación óptima no trivial ($k^* = 2$) y la alineación por centroide producen un contorno intermedio que captura bien la transición de forma entre el primer y el último slice con ET del caso. Es el caso donde la mejora de rotación se hace más evidente al comparar contra la versión sin pre-alineación.

Caso 4 — cruzado entre casos (0066 → 0070, $k^* = 24$). Limitación intrínseca del método. La rotación elevada ($k^* = 24$ sobre $n = 26$, equivalente a casi una vuelta completa) revela que no existe una correspondencia natural entre los vértices de contornos provenientes de casos distintos. La salida es un polígono simple sin auto-intersecciones, pero sin significado clínico: representa el promedio geométrico entre dos tumores no comparables.

VII-E. Limitación documentada

La interpolación lineal vértice a vértice asume implícitamente que los contornos están alineados espacialmente y representan instancias progresivas de una misma estructura. Las mejoras de Sección VII-B robustecen esta hipótesis para slices del mismo caso, pero no la sustituyen: cuando los contornos pertenecen a pacientes distintos o están muy desplazados, el método produce una salida geoméricamente válida — polígono simple, sin auto-intersecciones — pero clínicamente irrelevante. Métodos basados en *morphing*, distancia geodésica o transformadas de distancia con signo resolverían mejor este escenario y se consideran como extensión natural del módulo.

VII-F. Visualizaciones

Las Figuras 2 y 3 muestran los cuatro PNGs generados con la versión mejorada del interpolador. En cada gráfico se dibujan los contornos fuente A (azul punteado) y B (rojo punteado), y el contorno interpolado a $t = 0,5$ (verde sólido con marcadores).

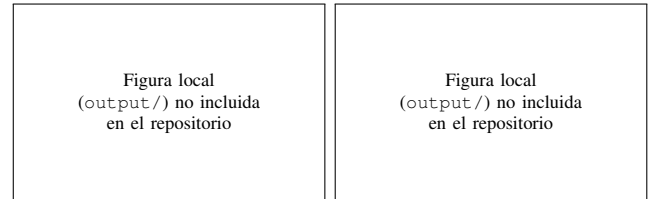


Figura 2. Caso *BraTS-GLI-00008-100* con las mejoras de pre-alineación activas. Izquierda: slices consecutivos 0070 → 0071, $k^* = 0$. Derecha: slices separados por 6 cortes, 0066 → 0072, $k^* = 1$. Ambos resueltos sin auto-intersecciones.

Cuadro II
EXACTITUD *dejar-un-corte-fuera* (110 CASOS, 5 190 CORTES/MÉTODO).

Método	Dice media	Dice mediana	% Dice > 0,8	Hausd. media (mm)	err. área mediana
Lineal	0,850	0,917	79,6 %	4,30	0,037
SDF	0,834	0,924	76,6 %	4,95	0,034
Spline	0,821	0,899	73,9 %	4,58	0,043

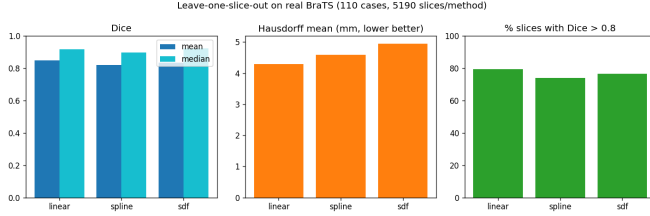


Figura 4. Comparación de métodos: solape (Dice/IoU, mayor es mejor), Hausdorff media (menor es mejor) y porcentaje de cortes con Dice > 0,8.

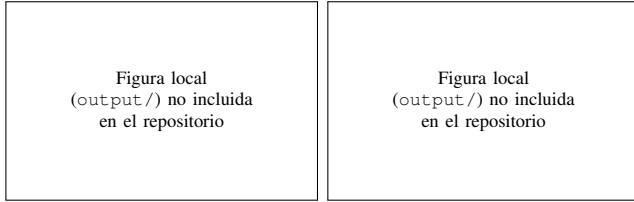


Figura 3. Izquierda: par extremo del caso *BraTS-GLI-00008-101*, primer (0070) contra último slice con ET (0072), $k^* = 2$ (mejor caso cualitativo). Derecha: par cruzado entre casos *BraTS-GLI-00008-100* (0066) y *BraTS-GLI-00008-101* (0070), $k^* = 24$ (limitación del método ante contornos no alineados espacialmente). Ambos resueltos sin auto-intersecciones.

El archivo de salida se escribe en formato `.obj` 3D con $z = 0,0$ por vértice, lo que asegura compatibilidad con visores estándar como ParaView, Online 3D Viewer e ImageToSTL.

VIII. VALIDACIÓN CUANTITATIVA CON DATOS REALES

VIII-A. Protocolo *dejar-un-corte-fuera*

Para comparar objetivamente los tres métodos se usa un protocolo de *dejar-un-corte-fuera* (*leave-one-slice-out*): para cada corte interior k se oculta el contorno real C_k , se predice a partir de los vecinos C_{k-1} y C_{k+1} con cada método y se compara la predicción $\hat{C}_k$ contra C_k . Como las regiones de ET presentan huecos en z , el parámetro se fija por las alturas reales, $t = (z_k - z_{k-1}) / (z_{k+1} - z_{k-1})$, descartando tríos con hueco > 3 cortes. Se evaluaron **110 casos** (los de mayor número de cortes ET entre los primeros 250 extraídos), con **5 190 cortes** ocultados por método; diez tumores se reconstruyeron además en 3-D.

VIII-B. Exactitud de la interpolación

La Tabla II resume la exactitud agregada y las Figuras 4–6 la ilustran.

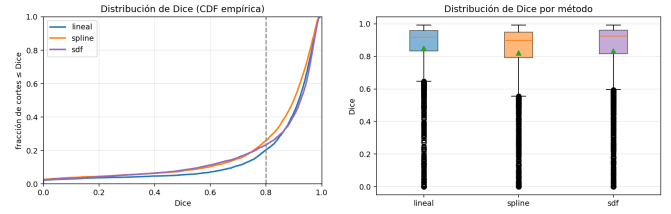


Figura 5. Distribución de Dice por método. Izquierda: CDF empírica. Derecha: diagrama de caja. La mediana ($\approx 0,92$) supera con creces a la media porque una cola de cortes difíciles puntúa cerca de cero.

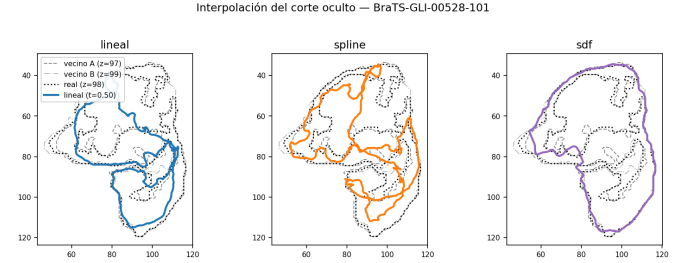


Figura 6. Predicción del corte oculto (vista 2-D) para *BraTS-GLI-00528-101*: vecinos *A* y *B* (gris), corte real (negro punteado) e interpolación de cada método. Lineal y SDF siguen de cerca el contorno real; el spline sobre-dispara.



Figura 7. Tumor *BraTS-GLI-02066-105*: verdad de terreno (izq.) y reconstrucción lineal por Poisson (der.); razón de volumen $\approx 1,02$.

VIII-C. Discusión de resultados

Los métodos **lineal** y **SDF** lideran (Dice mediana $\approx 0,92$, $\sim 80\%$ de cortes con Dice > 0,8), mientras que el **spline** queda por detrás. En datos reales la forma del tumor cambia de manera irregular entre cortes y el ajuste suave del spline sobre-dispara; los métodos de dos vecinos son más robustos. Esto es lo *contrario* al caso sintético suave, donde el spline ganaba: buena señal de que la métrica es sensible a la irregularidad real. La diferencia entre mediana y media revela una distribución sesgada: el corte típico es excelente, pero una minoría de cortes difíciles (extremos del tumor o regiones que se dividen) hunde la media.

En cuanto a volumen, para tumores sólidos la razón malla/vóxel está en 1,0–1,15 (p.ej. 00528-101: 97 646 vs 87 993 mm³, razón 1,11); cuando es mayor, Poisson rellena núcleos necróticos/anulares que la máscara de vóxeles deja huecos. La Figura 7 muestra una reconstrucción limpia frente a su verdad de terreno.

VIII-D. Visualizador interactivo

Se desarrolló un informe HTML autocontenido (`assets/report.html`) que combina las ecuaciones

de los métodos, gráficas interactivas de métricas y un **selector desplegable de tumores** con las superficies 3-D (verdad de terreno vs. lineal/spline/SDF) y la tabla de métricas por caso.

IX. DECISIONES DE DISEÑO Y PROBLEMAS RESUELTOS

IX-A. Compatibilidad C++17 $\rightarrow$ C++20

Durante la compilación inicial el proyecto fallaba con el error `'iter_value_t' is not a member of 'std'`. La causa es que `pujCGAL/SegmentsIntersection.hxx` usa `std::iter_value_t`, disponible únicamente desde C++20. Para no modificar la librería del profesor se agregó al `CMakeLists.txt` del módulo:

```
1 target_compile_features(  
2   contour_interpolator PRIVATE cxx_std_20  
3 );
```

Listing 4. Activación de C++20 sólo para el target del módulo.

Esto eleva el estándar exclusivamente para el ejecutable `contour_interpolator` y resuelve la compilación sin tocar código externo.

IX-B. Formato `.obj` 2D vs 3D

La primera versión del exportador escribía vértices en 2D ($v \times y$). Visores 3D como ParaView e ImageToSTL rechazaban el archivo con errores del tipo *“Invalid vertex index”* o *“Failed to import model”*. La solución fue introducir una función local `write_obj_3d` que agrega $z = 0,0$ a cada vértice, produciendo el formato estándar $v \times y \ 0.0$ compatible con cualquier visor 3D, sin necesidad de transformaciones posteriores.

IX-C. Ausencia de auto-intersecciones en contornos BraTS

Durante el desarrollo se ejecutó un barrido amplio de **94 pares** de slices (consecutivos, separados, extremos y cruzados entre casos), sin detectar ninguna auto-intersección. El análisis geométrico indica que, *en los pares evaluados*, los contornos ET resultaron aproximadamente convexos y morfológicamente suaves: el promedio ponderado de dos contornos convexos alineados por longitud de arco tiende a producir un contorno simple. No obstante, el tumor realzante puede presentar formas anulares o irregulares, por lo que esta observación se limita al conjunto probado y no se generaliza a todo el dataset. Las tres mejoras de pre-alineación descritas en Sección VII-B (normalización CCW, alineación por centroide y rotación óptima del punto de inicio) *preservan* esta propiedad: ninguna introduce nuevas aristas ni modifica la topología del contorno interpolado, sólo reasignan correspondencias o aplican rigid transforms (traslación, inversión, rotación cíclica). En consecuencia, los 4 pares del experimento final mantuvieron la validez topológica. El módulo `SelfIntersectionResolver` está correctamente implementado (detección con Bentley–Ottmann, partición en sub-lazos y selección por área máxima) pero no se activa con datos reales de BraTS. Para demostrar su funcionamiento de forma controlada se planea un experimento con correspondencia de vértices deliberadamente errónea (*modo stress*: rotación del array B o rotación geométrica previa), donde se garantiza la aparición de cruces.

IX-D. Independencia del módulo respecto al equipo

El módulo se diseñó para correr directamente sobre los `.obj` extraídos de BraTS, sin depender del output de los otros integrantes del equipo (Jesús: correspondencia de vértices por ángulo polar; Santiago: reconstrucción 3-D). La reconstrucción 3-D presentada en este informe (Sección V) se realiza con CGAL (`poisson_surface_reconstruction_delaunay`); una versión preliminar del plan contemplaba VTK, finalmente no utilizada. Esta decisión reduce el riesgo de integración y permite validar el componente de Abel en cualquier momento sin coordinación con el resto del equipo. La integración se realiza a posteriori a través del tipo común `TContour` y del formato `.obj`, ambos estables y bien definidos.

IX-E. Entorno CGAL 5.6 y dependencias

El proyecto se compila contra CGAL 5.6 (no 6.x), por lo que se usa `CGAL::AABB_traits` (no `AABB_traits_3`) y, al no existir `interpolated_corrected_curvatures`, la curvatura media se implementó a mano. Eigen —requerido por Poisson y `jet_estimate_normals`— no estaba instalado ni había sudo: al ser *header-only* se incorporó como usuario y se apuntó CMake con `-DEIGEN3_INCLUDE_DIR`. Además, `pujCGAL/SegmentsIntersection.hxx` usaba `std::get_if` sobre lo que en 5.6 es un `boost::variant`; se hizo portable según `CGAL_VERSION_NR` (`boost::get` en 5.x, `std::get_if` en ≥ 6.0).

IX-F. Convención de etiquetas BraTS

El tumor activo es la etiqueta 3, pero los primeros casos están post-tratamiento (sólo etiquetas 2 y 4) y casi no tienen ET; la extracción no producía contornos. El estudio selecciona los casos con más cortes de ET reales.

IX-G. Colapso de la reconstrucción y su solución

En pilas de anillos irregulares, `mst_orient_normals` descartaba la mayoría de los puntos y la malla de Poisson colapsaba (un caso pasó de 387 a 32 957 mm³ tras corregirlo). La solución fue *no* descartar esos puntos, sino asignarles como respaldo la normal exterior en el plano del contorno, fiable en la superficie lateral.

IX-H. Tumores muy pequeños (modo de fallo)

Tumores de pocos miles de mm³ tienen contornos demasiado escasos para una superficie de Poisson estable y se reconstruyen incompletos (Figura 8); se documentan como modo de fallo honesto.

IX-I. Escalabilidad

La validación lanza el binario una vez por corte: los 1 350 casos serían horas, por lo que el estudio (110 casos) se ejecutó en segundo plano; el dataset completo requeriría una validación en proceso, sin subproceso por corte.



Figura 8. Modo de fallo en un tumor muy pequeño *BraTS-GLI-02093-101*: la verdad de terreno (izq.) es válida, pero la reconstrucción lineal (der.) colapsa a un fragmento (~ 254 vs $3\,793\text{ mm}^3$): los contornos son demasiado escasos para Poisson.

X. CONCLUSIONES PARCIALES

- El pipeline corre de extremo a extremo sobre datos reales del dataset *BraTS 2024 GLI*, generando un contorno interpolado válido en formato `.obj` 3D que se renderiza en visores externos sin transformaciones adicionales.
- La librería `pujCGAL` provee el barrido de Bentley–Ottmann en una interfaz tipo *iterator*, lo cual evita reimplementar el algoritmo y permite concentrar el esfuerzo del módulo en la lógica de partición y selección de sub-lazos.
- Cada `Contour` clase (`ContourResampler`, `LinearInterpolator`, `SelfIntersectionResolver`) tiene una responsabilidad única y opera sobre el tipo común `TContour`. Esto hace al módulo independiente de los componentes desarrollados por los otros integrantes del equipo y permite integrarlo sin acoplamiento adicional.
- Como trabajo futuro inmediato se contempla evaluar el módulo con pares de slices que produzcan auto-intersecciones reales —típicamente en los cortes superiores e inferiores del tumor, donde la topología cambia bruscamente— y comparar la selección por área máxima contra estrategias basadas en orientación o en distancia de Hausdorff entre sub-lazos candidatos.
- Se integraron tres métodos de interpolación (lineal, spline y basado en forma) con reconstrucción 3-D por Poisson y un módulo de métricas, validados con un protocolo *dejar-un-corte-fuera* sobre 110 casos reales de BraTS.
- En datos reales, la interpolación **lineal** y la basada en **forma** (SDF) son las más fiables (Dice mediana $\approx 0,92$); el **spline** sobre-dispara ante cambios bruscos de forma. Las reconstrucciones son cerradas y su volumen coincide con la verdad de terreno salvo en tumores muy pequeños.
- Un visualizador HTML interactivo (selector de tumor, gráficas y superficies 3-D) acompaña al informe. Como trabajo futuro se plantea una validación en proceso para todo el dataset y el manejo de multifocalidad (varios contornos por corte).

REFERENCIAS

- [1] G. Barequet and M. Sharir, “Piecewise-linear interpolation between polygonal slices,” *Computer Vision and Image Understanding*, vol. 63, no. 2, pp. 251–272, 1996.
- [2] T. M. Lehmann, C. Gönner, and K. Spitzer, “Survey: interpolation methods in medical image processing,” *IEEE Transactions on Medical Imaging*, vol. 18, no. 11, pp. 1049–1075, 1999.
- [3] J. Souvion, “Correcting self-intersecting polygons using minimal memory,” *arXiv:1305.4573*, 2013.

- [4] U. Baid *et al.*, “The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification,” *arXiv:2107.02314*, 2021.
- [5] M. Kazhdan, M. Bolitho, and H. Hoppe, “Poisson surface reconstruction,” in *Proc. Eurographics Symposium on Geometry Processing*, 2006, pp. 61–70.
- [6] W. E. Lorensen and H. E. Cline, “Marching cubes: A high resolution 3D surface construction algorithm,” *ACM SIGGRAPH Computer Graphics*, vol. 21, no. 4, pp. 163–169, 1987.
- [7] M. Meyer, M. Desbrun, P. Schröder, and A. H. Barr, “Discrete differential-geometry operators for triangulated 2-manifolds,” in *Visualization and Mathematics III*, 2003, pp. 35–57.
- [8] S. P. Raya and J. K. Udupa, “Shape-based interpolation of multidimensional objects,” *IEEE Transactions on Medical Imaging*, vol. 9, no. 1, pp. 32–42, 1990.
- [9] G. T. Herman, J. Zheng, and C. A. Bucholtz, “Shape-based interpolation,” *IEEE Computer Graphics and Applications*, vol. 12, no. 3, pp. 69–79, 1992.
- [10] J. L. Bentley and T. A. Ottmann, “Algorithms for reporting and counting geometric intersections,” *IEEE Transactions on Computers*, vol. C-28, no. 9, pp. 643–647, 1979.
- [11] H. Blum, “A transformation for extracting new descriptors of shape,” in *Models for the Perception of Speech and Visual Form*, W. Wathen-Dunn, Ed. Cambridge, MA: MIT Press, 1967, pp. 362–380.